

An Energy-Efficient Parallelism Scheme for Deep Neural Network Training and Inferencing on Heterogeneous Cloud Resources

Hong-Ju Jeong
Kyung Hee University, Yongin-si
17104, South Korea
sub06038@khu.ac.kr

Hacksung Boo
Kyung Hee University, Yongin-si
17104, South Korea
hsb@khu.ac.kr

Jiseung Bae
Kyung Hee University, Yongin-si
17104, South Korea
n1mb0606@khu.ac.kr

Mincheol Jeon
Kyung Hee University, Yongin-si
17104, South Korea
2019102224@khu.ac.kr

Eui-Nam Huh*
Kyung Hee University, Yongin-si
17104, South Korea
johnhuh@khu.ac.kr

ABSTRACT

The emergence of Large Language Models (LLM) and generative AI has led to an explosive increase in computational demands across cloud computing data centers. The growing number of parameters in deep learning models results in significant power consumption problem, leading to the need for cost-effective and eco-friendly data centers. Furthermore, with the advent of multi-cloud environments, deep learning computations, not only for training but also for inference, no longer occur on a single hardware unit but are distributed across various heterogeneous hardware nodes forming clusters. In this paper, we present solutions to these challenges from a parallelism perspective. Considering the characteristics of the models, we implement data parallelism and model parallelism, partitioning models and data across heterogeneous hardware nodes for power-efficient learning and inferencing. To quantify the impact, we measured the power consumption of CPUs, GPUs, and RAM during the experiments, providing insights into the energy efficiency of the proposed partitioning strategies. Furthermore, we conducted a carbon footprint analysis, converting the measured power consumption into equivalent carbon emissions. The study highlights the necessity of partitioning research for energy-efficient learning and inferencing, addressing the identified issues.

CCS CONCEPTS

• Computing methodologies; • Parallel computing methodologies; Artificial intelligence; • Hardware; • Power and energy;

KEYWORDS

Parallelism, Deep learning, Heterogeneous, Power consumption, Carbon footprint

*Corresponding author : Eui-Nam Huh



This work is licensed under a Creative Commons Attribution International 4.0 License.

ICIIT 2024, February 23–25, 2024, Ho Chi Minh City, Vietnam

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1671-3/24/02

<https://doi.org/10.1145/3654522.3654596>

ACM Reference Format:

Hong-Ju Jeong, Hacksung Boo, Jiseung Bae, Mincheol Jeon, and Eui-Nam Huh. 2024. An Energy-Efficient Parallelism Scheme for Deep Neural Network Training and Inferencing on Heterogeneous Cloud Resources. In *2024 9th International Conference on Intelligent Information Technology (ICIIT 2024)*, February 23–25, 2024, Ho Chi Minh City, Vietnam. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3654522.3654596>

1 INTRODUCTION

Recent advances in hardware and the development of neural network models have ushered in a new era, characterized by the emergence of Large Language Models (LLMs) that leverage rich datasets. Foundation models such as GPT (Generative Pre-trained Transformer) and Llama (Large Language Model Meta AI) have demonstrated substantial improvements in accuracy across a multitude of Natural Language Processing (NLP) tasks [1].

However, these models face an overwhelming challenge due to the vast scale of datasets and an unprecedented increase in the number of model parameters. This naturally leads to a surge in computational demands, prompting the widespread deployment of data centers globally. The escalating power usage in these facilities contributes to increased carbon emissions, potentially exacerbating the impact of global warming [2].

There are two main approaches to reducing power consumption. The first is to enhance the performance of the hardware itself. This involves releasing products with lower energy consumption than existing hardware solutions to address the issue. However, this approach requires a significant investment of time and cost. The second approach involves modifying the software to utilize hardware more efficiently, eliminating the need for hardware replacement and allowing for substantial savings.

In this paper, we implement parallelism in software development, a well-known strategy for improving efficiency by distributing tasks. Adopting a parallelism perspective, we partition models and input data, considering their unique properties. For LLMs, characterized by an extensive number of model parameters, we partition models by layer or block and distribute them across diverse heterogeneous nodes. Given that the computational requirements of each layer or block may vary, distribution should align with the hardware's performance based on CPU/GPU FLOPs. Conversely, input batches for CNNs are dispersed across nodes, considering their relatively smaller model size.

This research proposes methods for the efficient implementation of parallelism with DNN models in heterogeneous nodes, with the goal of achieving energy-efficient learning and inference, considering hardware power consumption and heat generation.

2 BACKGROUND AND RELATED WORKS

2.1 Cost of LLM Training and Inference

Training Foundation Models like GPT and Llama entails the impractical utilization of thousands of GPUs over multiple months. These models operate within specialized clusters featuring high-speed, sophisticated networks, and accelerators. However, such clusters are expensive and challenging to acquire.

LLMs also require a considerable amount of computational power. PetaFLOPs, denoting 1,000 trillion floating-point operations per second, serve as a metric for computational intensity. For example, GPT-3-175B, equipped with 175 billion parameters, consumes approximately 3.6K PetaFLOPs-day during training. This incurs a cost of \$4 million when utilizing AWS on-demand instances [3].

The costs incurred during inference are also substantial. GPT-3 accumulates a cost of 2 cents per query, resulting in a monthly expense of \$7.5 billion with 100 million users making 10 queries daily. To mitigate these expenses, employing heterogeneous nodes for training and inference becomes imperative, allowing for cost reduction and efficient distributed computing.

2.2 Model Parallelism

2.2.1 Distributed Model Parallel Training. Traditional model-parallel training methods such as Megatron-LM [4] and DeepSpeed [5] focus on homogeneous node configurations within data center environments. However, due to the aforementioned cost-related issues, recent research has been actively exploring training and inference in distributed environments. DT-FM [3] proposed a study on training large Foundation Models in a distributed environment with heterogeneous networks featuring varying latency and bandwidth. The research introduced scheduling algorithms for allocating GPT-3 blocks to distributed nodes, considering different computational tasks during model training. This approach allowed for fast and efficient learning. However, it only considered heterogeneous networks, neglecting the consideration of heterogeneous nodes and overlooking the aspects of power consumption and temperature generation.

2.2.2 Distributed Model Parallel Inference. In contrast to training, where computing Optimizer States is essential, inference involves the straightforward loading of generated activations since only the forward pass is executed. Research endeavors like DADS [6] have explored employing additional CPU resources for inference. Building on this foundation, our study aims to distribute computations based on the performance of both CPU and GPU. This approach seeks to efficiently minimize energy consumption during inference.

2.3 Data Parallelism

2.3.1 Distributed Data Parallel (DDP). DDP is one of the distributed training packages supported by PyTorch, built on the torch distributed package. It operates across multiple nodes using multiple

processes, providing a solution to the Global Interpreter Lock (GIL) bottleneck, unlike data parallelism, which operates within a single process using multiple threads [7]. Each node is assigned a unique GPU ID, LOCAL_RANK, and a unique node ID, WORLD_RANK. During DDP execution, each node receives batches, conducts forward propagation, and computes local gradients. Subsequently, an all-reduce process is initiated across the cluster to aggregate all gradients, and the average gradient is computed by dividing it by the total number of nodes. The resulting average gradient is then utilized in the step process to update the model [8]–[10].

2.4 DRAM

2.4.1 Power Consumption Factors in DRAM. The power consumption of DRAM is significantly influenced by the following factors. Firstly, it is affected by the number of ones in a cache line being read or written. The power consumption increases noticeably as the number of ones in the cache line increases. Secondly, power consumption varies across different operations, with refresh operations exhibiting the highest power consumption, followed by substantial energy usage during read/write operations [11].

3 METHODS

3.1 Measurement of Model Computational Load

Measuring the computational load of a model enables a direct comparison among various models. Consequently, users can accurately evaluate whether the model they are employing is compatible with their respective environments. In this study, as inference and training occur in heterogeneous environments, we measure the computational load of the model's layers and blocks using hardware-independent metrics such as FLOPs and then distribute them across heterogeneous hardware.

For GPT, we can distribute it into transformer blocks and the logit layer in the head. The transformer block consists of an attention block and a Feed Forward network. Referring to the operations in Megatron-LM [6], when considering the number of transformer layers l , hidden size h , sequence length s , vocabulary size V , and training batch size B , the computational load of a transformer block is determined to be $96Bsh^2(1 + \frac{s}{6h})$.

The logit layer in the head transforms features of size h into vocabulary dimensions V . During the forward pass, it requires $2BshV$, and during back-propagation, an additional $4BshV$ is needed. Therefore, a total of $6BshV$ is necessary during the training process.

Once the computational load of each block and layer of the model is calculated, it is essential to distribute them according to the performance of each computational device. In this paper, we distribute layers based on the FLOPs ratio of the hardware. In Figure 1, you can also observe the model partitioning during inference.

3.2 CO₂Measurement Based on Power Consumption

Energy is estimated by measuring the power consumption of CPU, GPU, and DRAM, summing them up, and multiplying by the Power Usage Effectiveness (PUE), which takes into account additional energy required for cooling [12]. In this paper, we use the PUE coefficient of the global average for data centers in 2022, which is 1.55.

Table 1: Hardware specification of heterogeneous nodes

Node num	GPU	CPU	GFLOPs of CPU
Node 1	Nvidia GeForce RTX 2080 Ti #4	Intel(R) Xeon(R) Silver 4210 CPU @ 2.20GHz	153.6
Node 2	Nvidia GeForce RTX 2080 Ti #2	Intel(R) Xeon(R) Gold 5120 CPU @ 2.20GHz	537.6
Node 3	No GPU	Intel(R) Xeon(R) CPU E3-1240 v6 @ 3.70GHz	236.8
Node 4	No GPU	Intel(R) Xeon(R) CPU E3-1240 v6 @ 3.70GHz	236.8

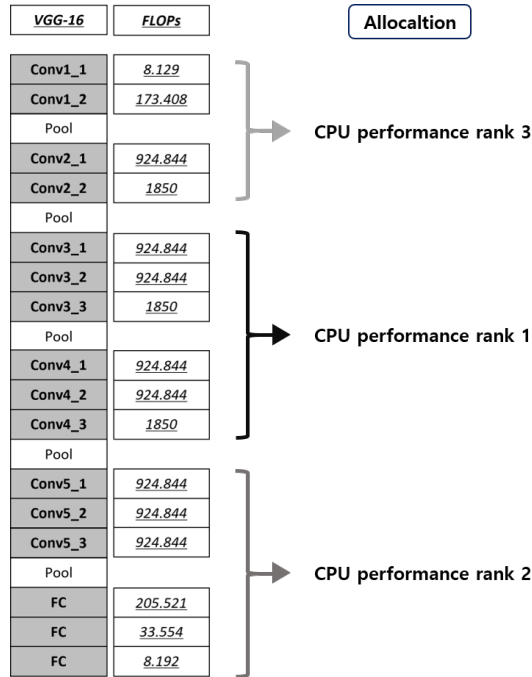


Figure 1: Layers allocation proportional to computation : We allocate layers based on the FLOPs per layer in VGGNet to hardware with an appropriate CPU.

The total power consumption is calculated as follows, where P_C is the total CPU power consumption, P_G is the total GPU power consumption, P_R is the total DRAM power consumption, and g is the number of GPUs used:

$$P = \frac{1.55 (P_C + P_R + gP_G)}{1000}$$

The energy is subsequently transformed into carbon dioxide emissions using the emission factor obtained from the EG-TIPS platform [13]. The conversion is expressed as:

$$CO_{2eq.} = 0.4434P$$

In this paper, we evaluate the model’s energy efficiency based on the calculated CO_2 emissions.

3.3 Memory Analysis

3.3.1 Impact of Model Parameter Loading on Memory Usage. To identify sections machine learning algorithm code where significant memory usage occurs, a torch profiler was employed. The code was divided into segments, and call stacks were traced to analyze memory usage. The analysis brought to light that noteworthy memory usage occurred in the following sections:

3.3.2 Memory Usage due to Batch Loading. The code responsible for invoking a pre-trained model exhibited high memory usage. Tracing the call stacks in this code segment unveiled the execution of functions dedicated to storing model parameters in the model constructor.

3.3.3 Issues and Solutions. Memory-intensive operations were noted in the code tasked with loading batches from the data loader. While the memory usage was less than that induced by model parameter loading, the existence of multiple batches within the data and the potential for escalated memory usage across epochs made the overall memory consumption noteworthy. Tracing the call stacks revealed that initial loading involved combining sample data for the batch, performing transformations (normalization, flipping, etc.), continuously allocating memory. Finally, memory was allocated for the batch size to inscribe the transformed and combined samples.

4 EXPERIMENT RESULTS

4.1 Experiment Settings

Hardware specifications of all nodes are in Table 1. “#” in the GPU column represents the number of GPUs. Temperature and CPU power measurements were acquired using “turboStat”, a Linux command-line tool that provides detailed information about the processor’s power and performance-related metrics. Measurements related to GPUs were obtained using “nvidia-smi”, the NVIDIA System Management Interface.

4.2 Experiment 1: Energy Efficiency Measurement during Model Partitioned Training

In this experiment, the GPT-2 model, comprising 12 transformer blocks, was partitioned and allocated to individual GPUs, completing 10 epochs. The GPT-2 model features a total of 124 million parameters, with h as 768, s as 1,024, V as 50,257, l as 12, and B as

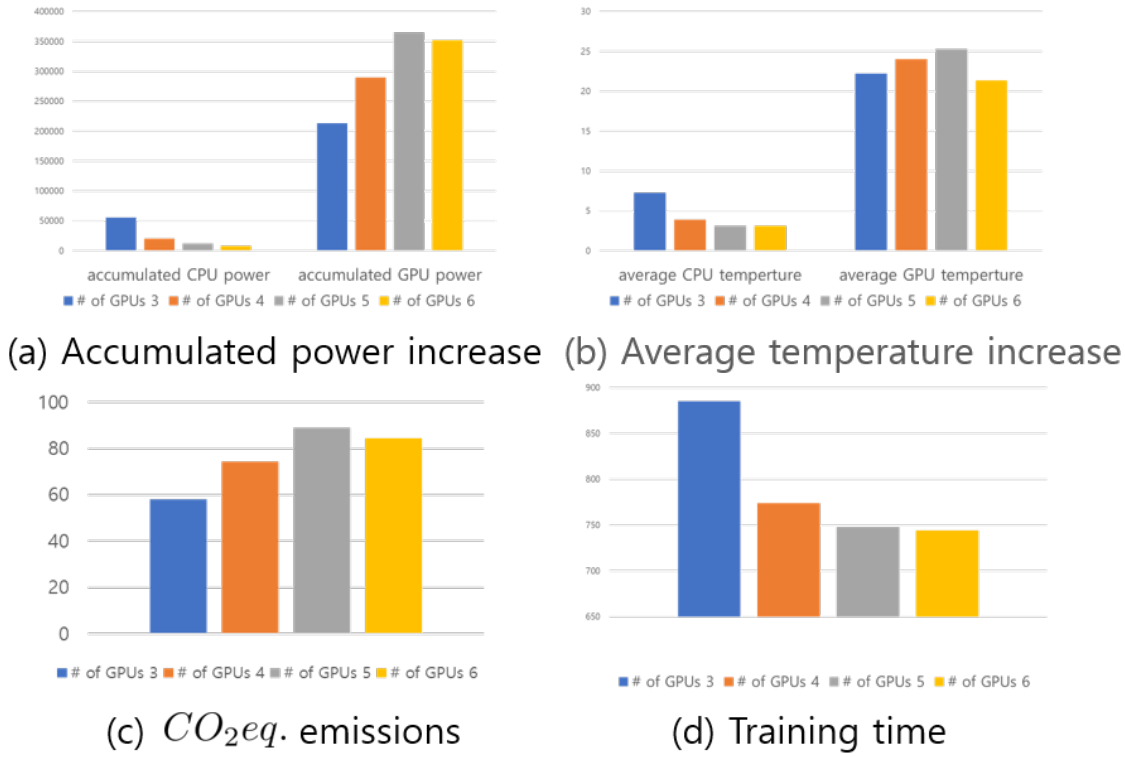


Figure 2: Energy measurement during model partitioned training : The number of GPUs ranges from 3 to 6, represented by colors from blue to yellow. (a) and (b) show CPU measurements on the left and GPU measurements on the right.

16. Each transformer block has 1.1 TeraFLOPs, and the head’s logit layer has 3.8 TeraFLOPs. The training was conducted on nodes with identical GPUs.

In Figure 2, the increase represents the disparity between the average power measured during training and the baseline average power measured during idle periods. As the number of GPUs allocated for model partitioning increases, the total cumulative power also escalates. However, specific points, such as the transition from 5 to 6 GPUs, exhibit a noticeable decrease in power consumption, leading to an overall reduction in carbon dioxide emissions. While the blue section (3 GPUs) boasts the lowest carbon dioxide emissions, it has the longest training time. This trade-off situation highlights the need for careful consideration, indicating the potential for adjustments based on specific user requirements.

4.3 Experiment 2: Memory Energy Consumption According to the Number of Nodes

In the laboratory setup, Node 1 and Node 2 have different memory sizes, with Node 2 having a memory size four times larger, resulting in inherently higher memory power consumption. As memory power consumption is heavily influenced by the memory capacity, it is necessary to normalize the comparison by adjusting to the same memory size to assess pure memory energy consumption accurately. Consequently, we computed the energy consumption

per unit of memory size for each scenario. The measured results, indicating the energy consumption for the same memory size, revealed that memory energy consumption in distributed nodes was higher. This discrepancy is attributed to the observation that the reduction in training time achieved through distributed learning did not lead to a proportionate decrease in memory consumption, despite adjustments for the same memory size.

In model training, the significance lies not only in the nature of the task but also in the speed at which the task is accomplished. In this study, we devised a metric to indicate how quickly learning occurs in each scenario.

$$Performance = \frac{NumberofEpochsCompleted}{TrainingTime}$$

Observations revealed that the use of distributed learning resulted in, on average, a 43% performance improvement compared to a single node. To better understand performance under equal energy allocations, we computed the Performance-to-Memory-Energy ratio as a metric. The results, presented in Table 2, suggest that, with an equivalent energy allocation, employing distributed learning yields superior performance.

Table 2: Performance Results of Experiment 2

Node	Performance	Ratio to DistributedNode (%)	Performance per Unit Energy	Ratio to DistributedNode (%)
Node 1	0.0562	144.1	0.0066	134.8
Node 2	0.0571	141.8	0.0082	108.5
Average	0.0566	143.0	0.0074	120.3

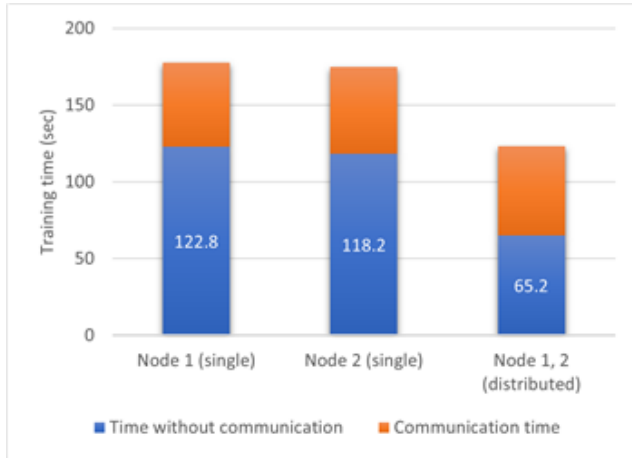


Figure 3: Execution Time with Different Node Counts in the Lab Environment

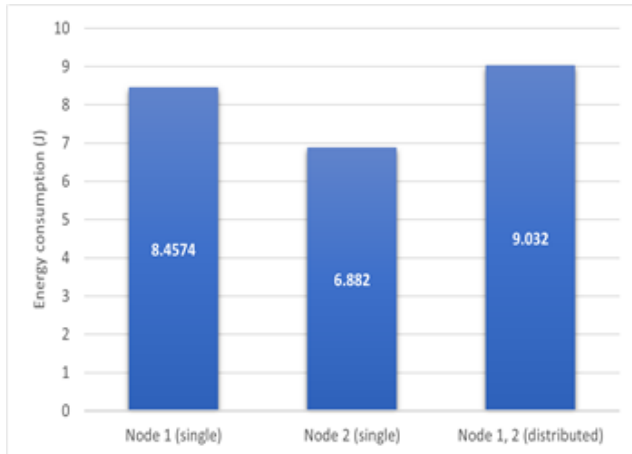


Figure 4: Memory Energy Usage per Node in the Lab Environment

4.4 Experiment 3: Energy Efficiency Measurement during Model Partitioned Inference

In this experiment, we partitioned the VGGNet16 model, creating a pipeline, and conducted 300 inferences. The VGG16 has 138.36 million parameters and a total of 15.5 GigaFLOPs. The experiment

compares the results of distributing layers with and without considering the FLOPs of each CPU.

In Figure 5, with an increase in the number of nodes, the inference time generally decreases, excluding the scenario with only one node. Although carbon dioxide emissions show a slight increase, the overall trend remains similar. This indicates that the slope of the operational gain trend, achieved by partitioning the model into a pipeline, is steeper than the slope of the carbon dioxide emission trend. Therefore, the optimal point for energy-efficient inference can be found at the intersection of these two slopes.

Additionally, in the experiment, when the number of nodes is 3 or 4, distributing layers equally resulted in lower carbon dioxide emissions compared to considering performance. However, with two nodes, carbon dioxide emissions were lower when considering performance of nodes. This is because the total FLOPs of VGGNet16 is relatively low, causing surplus power when there are three or more nodes. Models with higher computational loads would likely exhibit a more pronounced inverse relationship.

5 CONCLUSION

In conclusion, our study addresses the escalating computational demands posed by Large Language Models (LLMs) and generative AI, emphasizing the need for energy-efficient solutions in training and inference. With the proliferation of deep learning models and the surge in the number of parameters, power consumption in data centers has become a critical concern. Our research tackles this issue by proposing parallelism strategies, specifically data parallelism and model parallelism, tailored to heterogeneous hardware nodes.

We conducted comprehensive experiments to measure the impact of these parallelism strategies on energy efficiency, considering the power consumption of CPUs, GPUs, and RAM. The results demonstrate the effectiveness of our proposed approaches in achieving superior performance under equivalent energy allocations. Notably, our findings reveal a 43% average performance improvement when employing distributed learning, highlighting its potential for enhancing both learning and inference tasks.

Furthermore, we explored the carbon footprint associated with the power consumption of hardware during model training and inference. The study underscores the importance of considering energy-efficient partitioning strategies to mitigate environmental impacts, as evident in our analysis of carbon dioxide emissions.

In summary, our research contributes valuable insights into optimizing energy-efficient parallelism for heterogeneous systems in deep neural network training and inferencing. By addressing both performance and environmental considerations, our proposed

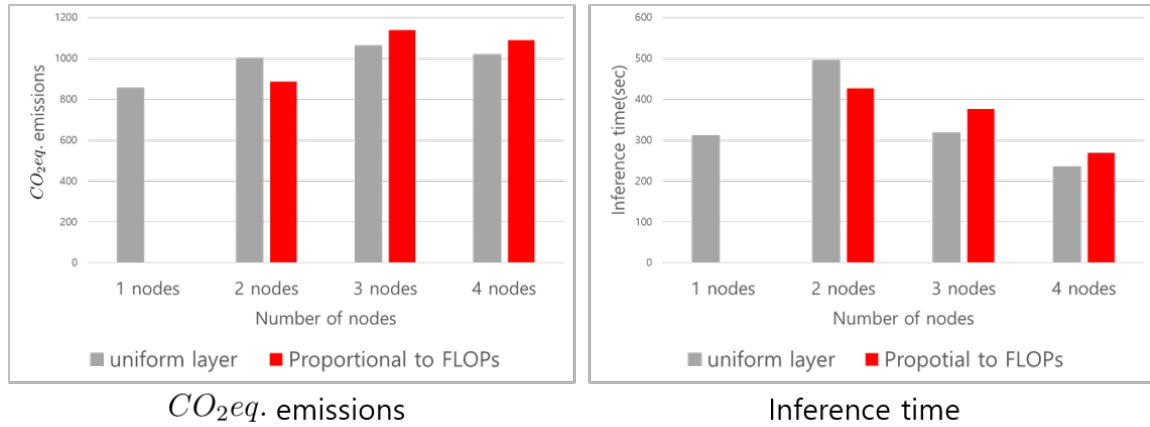


Figure 5: Energy measurement during model partitioned inference. Gray represents an equal distribution of layers regardless of node performance, while red represents a proportional distribution based on node performance.

strategies offer a promising avenue for the development of cost-effective and eco-friendly data centers in the era of expansive language models and generative AI.

ACKNOWLEDGMENTS

This research was supported by the MSIT(Ministry of Science and ICT), Korea, under the ITRC(Information Technology Research Center) support program(IITP-2023-RS-2023-00258649) supervised by the IITP(Institute for Information & Communications Technology Planning & Evaluation)

REFERENCES

- [1] Bommasani, Rishi, *et al.* 2021. On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258
- [2] HORNER, Nathaniel C.; SHEHABI, Arman; AZEVEDO, Inês L. 2016. Known unknowns: indirect energy effects of information and communication technology. *Environmental Research Letters*, 11.10: 103001
- [3] Yuan, Binhang, *et al.* 2022. Decentralized training of foundation models in heterogeneous environments. *Advances in Neural Information Processing Systems*, 35, 25464-25477
- [4] Narayanan, Deepak, *et al.* 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*
- [5] Conglong Li, Zhewei Yao, *et al.* 2022. DeepSpeed Data Efficiency: Improving Deep Learning Model Quality and Training Efficiency via Efficient Data Sampling and Routing. <https://www.microsoft.com/en-us/research/publication/deepspeed-data-efficiency-improving-deep-learning-model-quality-and-training-efficiency-via-efficient-data-sampling-and-routing/>
- [6] Chuang Hu, Wei Bao, Dan Wang, and Fengming Liu. 2019. Dynamic adaptive DNN surgery for inference acceleration on the edge. *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*, IEEE
- [7] Sam Gross. 2023. PEP 703 – Making the Global Interpreter Lock Optional in CPython. <https://peps.python.org/pep-0703/>
- [8] Shen Li, Yanli Zhao, *et al.* 2020. PyTorch distributed: experiences on accelerating data parallel training. *Proc. VLDB Endow.* 13, 12 (August 2020), 3005–3018
- [9] Shen Li. GETTING STARTED WITH DISTRIBUTED DATA PARALLEL. https://pytorch.org/tutorials/intermediate/ddp_tutorial.html
- [10] DISTRIBUTED DATA PARALLEL. Pytorch Docs. <https://pytorch.org/docs/master/notes/ddp.html>
- [11] Saugata Ghose, Abdullah Giray Yaglikci, *et al.* 2018. What Your DRAM Power Models Are Not Telling You: Lessons from a Detailed Experimental Study. *Proc. ACM Meas. Anal. Comput. Syst.* 2, 3, Article 38 (December 2018), 41 pages
- [12] Strubell, Emma, Ananya Ganesh, and Andrew McCallum. 2019. Energy and policy considerations for deep learning in NLP. arXiv preprint arXiv:1906.02243
- [13] EG-TIPS: Energy Greenhouse Gas Comprehensive Information Platform. https://tips.energy.or.kr/carbon/Ggas_statistics03.do